In-Class C++ Programming Test (EGC 211P Section A)

September 19, 2025

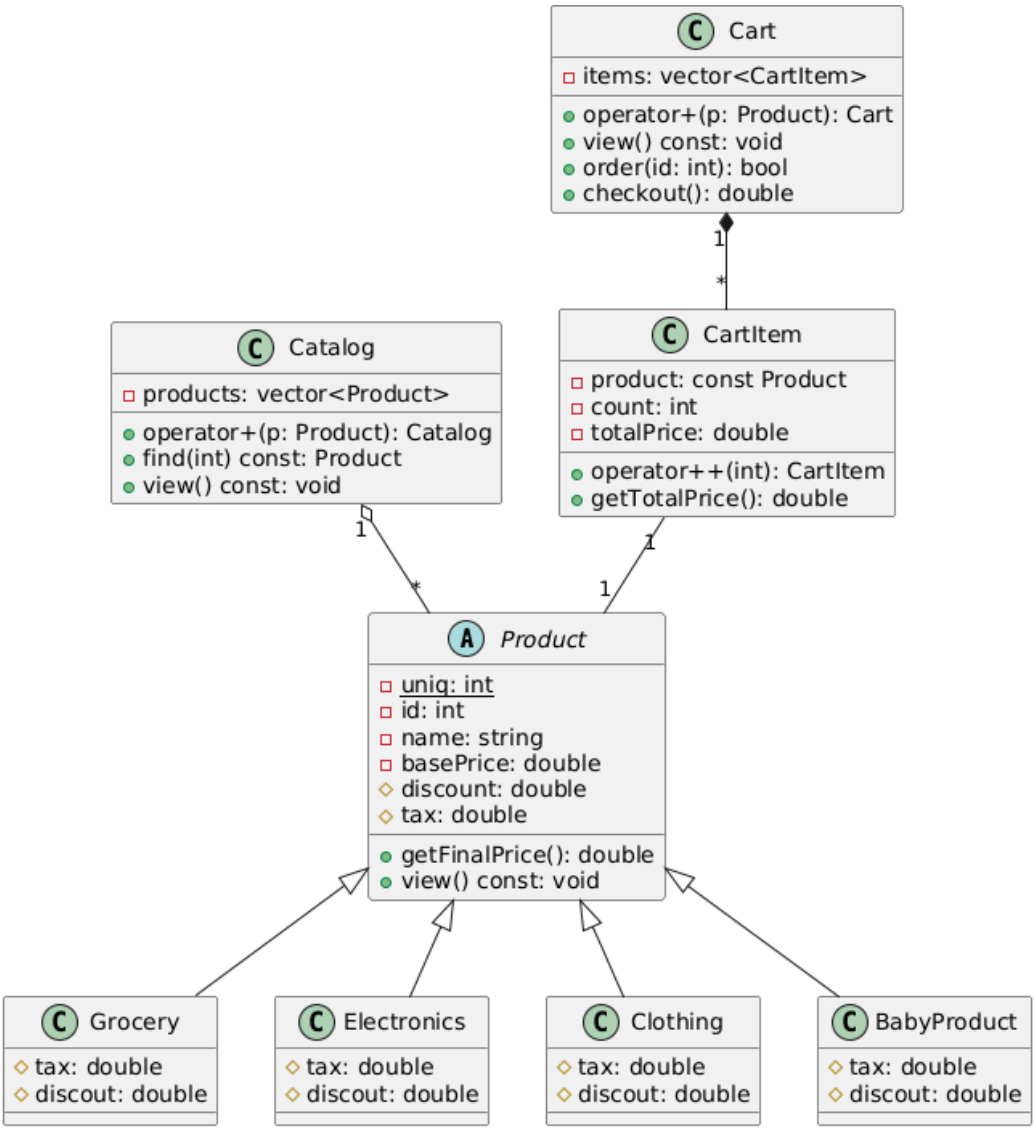
Duration: 90 minutes (9:15am – 10:45am)

Code should be submitted to DOMJudge

## Files to be submitted: classes.h, classes.cpp, main.cpp

**Problem:** Design a component of e-commerce platform, which enables the vendors to add products in a catalog for sale, enables the customers to view the catalog and choose products to place-order and then to checkout for purchase. For simplicity, one single user will act as the vendors as well as the customers for this case.

The specific entities for the design are as follows:



1. Product:
    - represents product for sale, has details such as name (string), id (int), base price, discount, and tax. The price related attributes are of type double.
    - The company sells four types of products: *grocery, electronics, clothing, and baby products*.
    - Each type differs in base price, discount and tax from the others.
        - **Baby products: 5% tax, 5% discount**
        - **Electronics: 18% tax, 12% discount**
        - **Clothing: 18% tax, 0% discount**
        - **Grocery: 0% tax, 0% discount**
    - Each product has different id.
    - *Two instances of the same product means two different product objects those have the same name but different id. Use static variable to implement the unique id, starting from 0.*
    - **Final price (double) formula:**
        - **base price + base price * tax – base price * discount**
    - **Note that, tax and discounts are in %.**
2. Catalog:
    - has the list of products in it.
    - User should be able to add products in the catalog using add command as follows, with no output, ***which needs to implemented using overloaded + operator in the Catalog class***:
        - **`add <product_type> <product_name> <base_price>`**
    - Sample add commands:
        - `add grocery basmati 368`
        - `add clothing cotton_green_dress 1200`
        - `add babyproduct cerelac 339`
        - `add electronic JBL_headphone 6000`
    - Catalog can be viewed by the vendor as well as customers using catalog command as follows:
        - **`catalog`**
        - Output should be:

```
basmati 0 368

cotton_green_dress 1 1416

cerelac 2 339
```

```
JBL_headphone 3 6360
```

3. Cart:
    - represent shopping cart, where the products chosen by the customer are saved before checkout.
    - Customer should be able to add a specific to a cart using the `order` command as follows, with no output:
        - **`order <product_id> <count>`**
    - Sample order commands:
        - `order 0 5`
        - `order 2 10`
        - `order 0 1`
    - ***Order command should use the overloaded + operator in the Cart class***.
    - View Cart functionality should be present using the `cart` command, which should list all the cart items, as follows:
        - **`cart`**
        - Output should be:

          ```
          6 0 Basmati 2208
          10 2 cerelac 678
          ```
    - Checkout functionality is supported by Cart, using the `checkout` command. On checkout, the total price of the ordered items are shown as follows:
        - `checkout`
        - Output should be:

          ```
          2286
          ```
4. CartItem:
    - when a product is added to a cart, it becomes a cart item.
    - CartItem has two fields: count and total price.
    - If multiple products of the same type and name are added to the cart, then the cart has one instance of cartItem with count 2. **This operation needs to be implemented using the post-increment operator ++ in the CartItem class.**
5. Finally, your program should exit on `exit` command, producing no output.

- Output should be done only through the view methods of the relevant classes.
- The design is similar to the above UML. The listed member attributes and methods are mandatory in your code. You can add more members if you like. The meanings of the graphic notations used in the picture are:

  Red square: private

  Yellow diamond: protected

  Green circle: public

  Underlined member: static

- You are free to use reference or pointers for the parameter/return types specified in the UML.

E.g. *you can have*

*Catalog& operator+(Product& p) or Catalog* operator(Product& p)*

*instead of*

*Catalog operator+(Product p), which is given in the UML design.*

- Note that, all the price calculations should use double datatype, and should not round off the values.
- Make constructors/destructors as per your need.
- Only use `cin` for command parsing. Getline is not allowed.
- Libraries: iostream, string, vector
- Commands are assumed always to be correct. For wrong command options, program should ignore them, should not crash. No need to check for wrong commands and arbitrary number of command options. E.g.
  - `add books feynman_lectures 1200` // books is not recognized product category
  - `order 223 6` // 223 is a non-existent product ID
- Your program should be able to reproduce the following samples:

**Sample1:**

*cart*
*catalog*
*add babyproduct seat 1000*

*add babyproduct cap 20*
*catalog*
seat 0 1000
cap 1 20
*order 1 5*
*cart*
5 1 cap 100
*order 0 3*
*cart*
5 1 cap 100
3 0 seat 3000
*checkout*
3100
*exit*

**Sample2:**

```
add grocery basmati 368
add grocery basmati 378
add electrical LED_while_bulb 75
add grocery Mogra 448
catalog
basmati 0 368
basmati 1 378
Mogra 2 448
cart
order 45 5
cart
order 2 10
order 2 10
order 10 2
cart
20 2 Mogra 8960
checkout
8960
exit
```